Viva & Defence Study Guide

Security and Privacy for Machine Learning (216H17C701)

Understand & Defend Your Project

Gradient-Inversion Attack & Privacy Defences in Federated Learning

Name: Suyash Humne

Roll No.: 16014223085

Purpose: Internal preparation — concepts, rationale, Q&A

Viva Study Guide

This is your private preparation document. It explains every concept in the project in plain language, why each design choice was made, and the questions an examiner is likely to ask — with answers you can give in your own words. Read it top to bottom once, then re-read Section 6.

> **Your 30-second pitch.** "Federated learning claims your data stays private because you only share gradients, not data. I showed that's false: I built an attack where a curious server reconstructs a client's private image, pixel by pixel, from just the shared gradient — at 24.53 dB. Then I tested the four privacy defences from our syllabus — differential privacy, local DP, compressive privacy, and anonymity — against that same attack and measured how much privacy each buys and what accuracy it costs."

1 The Big Picture

Federated learning (FL). Instead of sending raw data to a server to train a model, each device keeps its data, computes the *gradient* of the model's loss on its local data, and sends only that gradient. The server adds up everyone's gradients to improve the shared model. Selling point: "your data never leaves your device."

The attack in one line. A gradient is computed *from* the data, so it carries information *about* the data. If you can run the computation backwards, you get the data back. That backwards step is "gradient inversion."

Why it matters. Hospitals, phones, and banks are being sold FL as "privacy-preserving." If gradients leak the data, that promise is broken — which is exactly why the defences in Modules 2–5 exist.

2 Core Concepts (plain language)

2.1 What is a gradient, and why does it leak?

The gradient tells you, for every weight in the network, which direction to nudge it to reduce the error on the current example. Because the input image flows through the network to produce that error, the gradient is a detailed, differentiable fingerprint of the input. The attacker fixes the network weights and searches for a fake image whose gradient matches the real one — when the gradients match, the fake image matches the real image.

2.2 How the attack works (the three tricks)

- **Cosine matching (not L2).** We compare the *direction* of the two gradients, not their raw distance. Direction is magnitude-independent, which makes the attack far more stable. (This is the key idea of Geiping et al. 2020, "Inverting Gradients".)
- **Total variation (TV).** A gentle penalty that keeps neighbouring pixels similar, so the reconstruction looks like a real image instead of random noise.
- **iDLG label trick.** We don't have to guess the label. For a single example, the true class is the one whose final-layer gradient has a negative sign. We read it off directly — 100% accurate for one image.

2.3 How we measure reconstruction quality

Metric	Meaning	Good value
MSE	Average squared pixel error	lower

PSNR (dB)	Pixel fidelity; >30 dB looks almost identical	higher
SSIM	Structural similarity (shapes/edges), 0–1	higher (→1)
LPIPS	Perceptual distance using a neural net; matches human judgement	lower

For a **defence**, you want the opposite: low PSNR/SSIM and high LPIPS mean the reconstruction is bad, i.e. privacy is protected.

3 The Five Modules — What Each Means and Where It Appears

Module 1 — Privacy Considerations in ML

The study of threats and attacks on ML systems. **In our project:** this is the attack itself and the *threat model* — we assume an "honest-but-curious" server: it follows the protocol but tries to learn what it can from what it legitimately receives (the gradient).

Module 2 — Differential Privacy (DP)

Idea: add carefully calibrated random noise so that the output barely changes whether or not any one person's data was included. Formally, a mechanism is (ε, δ)-DP if including/excluding one record changes output probabilities by at most a factor of e^{ε}. Small ε = strong privacy. **DP-SGD** applies this to training: clip each example's gradient to a maximum norm, then add Gaussian noise. **In our project:** this is the main defence. Clipping + noise destroys the fine detail the attack needs, and it comes with a provable guarantee. We sweep the noise level and read off the ε.

Module 3 — Local Differential Privacy (LDP)

Difference from DP: in central DP you trust the server to add noise; in **local** DP you trust no one, so *each client* adds noise to its own data before sending anything. Stronger privacy model, but more total noise, so more accuracy is lost. **In our project:** we apply per-client noise to the gradient before upload and show it protects more but costs more utility than central DP.

Module 4 — Privacy-Preserving Data Mining (k-anonymity)

Idea: before publishing data, make each record indistinguishable from at least k−1 others (k-anonymity), so no single person stands out. **In our project:** federated aggregation is the analogue — the server sees the *sum* of many clients' gradients (secure aggregation), so any one client is "hidden among k". We analyse why this helps but isn't enough on its own (a malicious server can still single someone out).

Module 5 — Compressive Privacy

Idea: protect privacy by *compressing* / reducing the dimensionality of what you share, discarding information an attacker would need while keeping enough for the task. **In our project:** two mechanisms — **gradient pruning** (send only the largest values, zero the rest) and **quantization** (send fewer bits per value). We sweep how aggressively we compress and plot privacy vs the "accuracy of the utility task" (the exact phrase from the syllabus).

4 The Second Attack: Membership Inference (LiRA)

Question it answers: "Was this specific person's record used to train the model?" Even that yes/no leaks privacy (e.g. "this patient was in the diabetes study"). **LiRA** (Carlini 2022) trains many "shadow" models to learn how a model behaves on data it did/didn't see, then runs a likelihood-ratio test. **Key metric:** TPR at a very low FPR — "can the attacker catch *some* members with near-certainty", which is the real-world harm, instead of average accuracy. **Why we include it:** it gives a hard number for leakage and lets us *audit* differential privacy — if DP works, this attack should drop to near chance.

5 Design Decisions (be ready to justify these)

Choice	Why
LeNet (small CNN)	Fits a 4 GB GPU; standard in the gradient-leakage literature; reconstructs cleanly.
Sigmoid, not ReLU	Smooth activations give smoother gradients, which invert more faithfully.
Untrained network	Fresh weights leak the most — the honest worst case, and the clearest demo.
Cosine loss	Direction-based, magnitude-invariant, stable; beats the original L2 method.
Restarts + median	Reconstruction is seed-sensitive; consensus over restarts stabilises it.
CIFAR-10	Public, non-sensitive, reproducible proxy for private image data — ethical by design.

6 Likely Viva Questions & Model Answers

Q1. In one sentence, what did you do?

I showed that federated learning's "only gradients are shared" promise can be broken by reconstructing private images from gradients, and I measured how well our syllabus's privacy defences stop that.

Q2. If no data is sent, how can the server recover the image?

Because the gradient is computed *from* the image and the known weights, so it encodes the image. I fix the weights and optimise a fake image until its gradient matches the real one — at which point the fake equals the real.

Q3. Why cosine similarity and not Euclidean distance?

Cosine compares direction and ignores magnitude. As a model trains, gradient magnitudes shrink and vary a lot, which destabilises L2 matching; cosine stays stable. It's the core improvement of the Inverting-Gradients method.

Q4. How do you recover the label without the data?

The iDLG trick: for a single example with cross-entropy, the true class is the one whose row in the last layer's weight-gradient has negative sign. It's exact for one image, so I don't optimise the label at all.

Q5. How does differential privacy stop the attack?

DP-SGD clips each gradient and adds Gaussian noise, which bounds how much any one example can affect the gradient. Since the attack feeds on exactly that per-example signal, the noise blurs the reconstruction — and it's provable: you get an (ε, δ) guarantee.

Q6. Difference between DP and Local DP?

Central DP trusts the server to add the noise; local DP trusts no one, so each client adds its own noise before sending. LDP is stronger on trust but needs more noise overall, so it costs more accuracy. I show both on the same plot.

Q7. What is the privacy–utility trade-off?

More privacy (more noise / more compression) means worse reconstruction for the attacker but also lower model accuracy. The interesting region is the "knee" where you block the attack while barely losing accuracy. My sweeps plot exactly this.

Q8. Why is TPR@low-FPR the right membership-inference metric?

Average accuracy can look fine while the attack still identifies a few people with certainty — and that certainty is the actual harm. TPR at low FPR measures whether the attacker can confidently catch *any* member, which is what privacy should prevent.

Q9. Where does compressive privacy fit?

Gradient pruning and quantization both reduce the information in the shared gradient — dimensionality/precision reduction. That's compressive privacy: throw away what the attacker needs while keeping enough for training.

Q10. Is this a real attack or a simulation?

It's a real implementation of the attack, run as a single-host simulation of one federated-learning round with an explicit client/server split. The maths and code are the genuine attack; the "network" is simulated locally so no real system or data is touched.

Q11. What's your individual contribution?

I built the whole pipeline from scratch and, more importantly, assembled a single benchmark that pits all four syllabus defence families against one attack with consistent metrics — a unified comparison I didn't find pre-existing.

Q12. Biggest limitation?

Small model and low-resolution data, and the honest-but-curious assumption. Absolute numbers would change at scale, and malicious-server attacks are stronger — which I discuss and cross-validate against the *breaching* framework.

Q13. How would you make the attack stronger?

Add a BatchNorm network and match BN statistics (GradInversion), use a generative image prior, or move to a malicious-server analytic attack — each is in my references and future-work.

Q14. How would you make the defence stronger without losing accuracy?

Combine defences (compress then add DP noise), tune the clipping norm, or use secure aggregation so the server only ever sees sums — raising the anonymity set.

7 What the Recording Shows (so you can narrate it)

1. The setup: client has a private image; only the gradient is uploaded.
2. The server recovers the label analytically.
3. The reconstruction optimisation runs — the image emerges from noise.
4. Side-by-side reveal: original vs reconstruction, with the metrics.
5. The defences: privacy–utility curves showing each one degrading the attack.

If asked to run it live, the command is `python src/federated_demo.py`; the defence sweep is `python src/run_defense_sweep.py`.